

AUTOMATION

Run Claude Code programmatically

Use the Agent SDK to run Claude Code programmatically from the CLI, Python, or TypeScript.

The **Agent SDK** gives you the same tools, agent loop, and context management that power Claude Code. It's available as a CLI for scripts and CI/CD, or as **Python** and **TypeScript** packages for full programmatic control.

To run Claude Code in non-interactive mode, pass `-p` with your prompt and any **CLI options**:

```
claude -p "Find and fix the bug in auth.py" --allowedTools
"Read,Edit,Bash"
```

This page covers using the Agent SDK via the CLI (`claude -p`). For the Python and TypeScript SDK packages with structured outputs, tool approval callbacks, and native message objects, see the **full Agent SDK documentation**.

Basic usage

Add the `-p` (or `--print`) flag to any `claude` command to run it non-interactively. All **CLI options** work with `-p` , including:

- `--continue` for **continuing conversations**
- `--allowedTools` for **auto-approving tools**
- `--output-format` for **structured output**

This example asks Claude a question about your codebase and prints the response:

```
claude -p "What does the auth module do?"
```

Start faster with bare mode

Add `--bare` to reduce startup time by skipping auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md. Without it, `claude -p` loads the same context an interactive session would, including anything configured in the working directory or `~/.claude`.

Bare mode is useful for CI and scripts where you need the same result on every machine. A hook in a teammate's `~/.claude` or an MCP server in the project's `.mcp.json` won't run, because bare mode never reads them. Only flags you pass explicitly take effect.

This example runs a one-off summarize task in bare mode and pre-approves the Read tool so the call completes without a permission prompt:

```
claude --bare -p "Summarize this file" --allowedTools "Read"
```

In bare mode Claude has access to the Bash, file read, and file edit tools. Pass any context you need with a flag:

To load	Use
System prompt additions	<code>--append-system-prompt</code> , <code>--append-system-prompt-file</code>
Settings	<code>--settings <file-or-json></code>
MCP servers	<code>--mcp-config <file-or-json></code>
Custom agents	<code>--agents <json></code>
A plugin	<code>--plugin-dir <path></code> , <code>--plugin-url <url></code>

Bare mode skips OAuth and keychain reads. Anthropic authentication must come from `ANTHROPIC_API_KEY` or an `apiKeyHelper` in the JSON passed to `--settings`. Bedrock, Vertex, and Foundry use their usual provider credentials.

 `--bare` is the recommended mode for scripted and SDK calls, and will become the default for `-p` in a future release.

Background tasks at exit

If Claude starts a background Bash task during a `claude -p` run, for example a dev server or a watch build, that shell is terminated about five seconds after Claude has returned its final result and

stdin has closed. The grace period lets a task that finishes right after the result still deliver its output. Before v2.1.163, a never-exiting background process would hold the `claude -p` invocation open indefinitely.

Background **subagents** and workflows are exempt from the five-second grace because their result is part of the final output, so `claude -p` waits for them to complete. From v2.1.182, that wait is capped at ten minutes by default so a stuck background agent cannot hold the process open indefinitely. Adjust the cap with `CLAUDE_CODE_PRINT_BG_WAIT_CEILING_MS`, or set it to `0` to wait without a limit.

Examples

These examples highlight common CLI patterns. For CI and other scripted calls, add `--bare` so they don't pick up whatever happens to be configured locally.

Pipe data through Claude

Non-interactive mode reads stdin, so you can pipe data in and redirect the response out like any other command-line tool.

This example pipes a build log into Claude and writes the explanation to a file:

```
cat build-error.txt | claude -p 'concisely explain the root cause
of this build error' > output.txt
```

With `--output-format json`, the response payload includes `total_cost_usd` and a per-model cost breakdown, so scripted callers can track spend per invocation without consulting the **usage dashboard**.

❗ As of Claude Code v2.1.128, piped stdin is capped at 10MB. If you exceed the cap, Claude Code exits with a clear error and a non-zero status. To work with larger inputs, write the content to a file and reference the file path in your prompt instead of piping it.

Add Claude to a build script

You can wrap a non-interactive call in a script to use Claude as a project-specific linter or reviewer.

This `package.json` script pipes the diff against `main` into Claude and asks it to report typos. Piping the diff means Claude doesn't need Bash permission to read it, and the escaped double

quotes keep the script portable to Windows:

```
{
  "scripts": {
    "lint:claude": "git diff main | claude -p \"you are a typo
linter. for each typo in this diff, report filename:line on one
line and the issue on the next. return nothing else.\""
  }
}
```

Get structured output

Use `--output-format` to control how responses are returned:

- `text` (default): plain text output
- `json` : structured JSON with result, session ID, and metadata
- `stream-json` : newline-delimited JSON for real-time streaming

This example returns a project summary as JSON with session metadata, with the text result in the `result` field:

```
claude -p "Summarize this project" --output-format json
```

To get output conforming to a specific schema, use `--output-format json` with `--json-schema` and a **JSON Schema** definition. The response includes metadata about the request (session ID, usage, etc.) with the structured output in the `structured_output` field.

This example extracts function names and returns them as an array of strings:

```
claude -p "Extract the main function names from auth.py" \
  --output-format json \
  --json-schema '{"type":"object","properties":{"functions":
{"type":"array","items":{"type":"string"}}},"required":
["functions"]}'
```

💡 Use a tool like **jq** to parse the response and extract specific fields:

```
# Extract the text result
claude -p "Summarize this project" --output-format json |
jq -r '.result'

# Extract structured output
claude -p "Extract function names from auth.py" \
  --output-format json \
  --json-schema '{"type":"object","properties":
{"functions":{"type":"array","items":
{"type":"string"}}},"required":["functions"]}' \
  | jq '.structured_output'
```

Stream responses

Use `--output-format stream-json` with `--verbose` and `--include-partial-messages` to receive tokens as they're generated. Each line is a JSON object representing an event:

```
claude -p "Explain recursion" --output-format stream-json --
verbose --include-partial-messages
```

The following example uses **jq** to filter for text deltas and display just the streaming text. The `-r` flag outputs raw strings (no quotes) and `-j` joins without newlines so tokens stream continuously:

```
claude -p "Write a poem" --output-format stream-json --verbose --
include-partial-messages | \
  jq -rj 'select(.type == "stream_event" and .event.delta.type?
== "text_delta") | .event.delta.text'
```

When an API request fails with a retryable error, Claude Code emits a `system/api_retry` event before retrying. You can use this to surface retry progress or implement custom backoff logic.

Field	Type	Description
<code>type</code>	<code>"system"</code>	message type

Field	Type	Description
subtype	"api_retry"	identifies this as a retry event
attempt	integer	current attempt number, starting at 1
max_retries	integer	total retries permitted
retry_delay_ms	integer	milliseconds until the next attempt
error_status	integer or null	HTTP status code, or null for connection errors with no HTTP response
error	string	error category: authentication_failed , oauth_org_not_allowed , billing_error , rate_limit , overloaded , invalid_request , model_not_found , server_error , max_output_tokens ,Or unknown
uuid	string	unique event identifier
session_id	string	session the event belongs to

The `system/init` event reports session metadata including the model, tools, MCP servers, and loaded plugins. It is the first event in the stream unless `CLAUDE_CODE_SYNC_PLUGIN_INSTALL` is set, in which case `plugin_install` events precede it. Use the plugin fields to fail CI when a plugin did not load:

Field	Type	Description
plugins	array	plugins that loaded successfully, each with <code>name</code> and <code>path</code>
plugin_errors	array	plugin load-time errors, each with <code>plugin</code> , <code>type</code> , and <code>message</code> . Includes unsatisfied dependency versions and <code>--plugin-dir</code> load failures such as a missing path or invalid archive. Affected plugins are demoted and absent from <code>plugins</code> . The key is omitted when there are no errors

When `CLAUDE_CODE_SYNC_PLUGIN_INSTALL` is set, Claude Code emits `system/plugin_install` events while marketplace plugins install before the first turn. Use these to surface install progress in your own UI.

Field	Type	Description
type	"system"	message type
subtype	"plugin_install"	identifies this as a plugin install event

Field	Type	Description
status	"started" , "installed" , "failed" , or "completed"	started and completed bracket the overall install; installed and failed report individual marketplaces
name	string, optional	marketplace name, present on installed and failed
error	string, optional	failure message, present on failed
uuid	string	unique event identifier
session_id	string	session the event belongs to

For programmatic streaming with callbacks and message objects, see [Stream responses in real-time](#) in the Agent SDK documentation.

Auto-approve tools

Use `--allowedTools` to let Claude use certain tools without prompting. This example runs a test suite and fixes failures, allowing Claude to execute Bash commands and read/edit files without asking for permission:

```
claude -p "Run the test suite and fix any failures" \
  --allowedTools "Bash,Read,Edit"
```

To set a baseline for the whole session instead of listing individual tools, pass a **permission mode**. `dontAsk` denies anything not in your `permissions.allow` rules or the **read-only command set**, which is useful for locked-down CI runs. `acceptEdits` lets Claude write files without prompting and also auto-approves common filesystem commands such as `mkdir` , `touch` , `mv` , and `cp` . Other shell commands and network requests still need an `--allowedTools` entry or a `permissions.allow` rule, otherwise the run aborts when one is attempted:

```
claude -p "Apply the lint fixes" --permission-mode acceptEdits
```

Create a commit

This example reviews staged changes and creates a commit with an appropriate message:

```
claude -p "Look at my staged changes and create an appropriate
commit" \
  --allowedTools "Bash(git diff *),Bash(git log *),Bash(git
status *),Bash(git commit *)"
```

The `--allowedTools` flag uses **permission rule syntax**. The trailing `*` enables prefix matching, so `Bash(git diff *)` allows any command starting with `git diff`. The space before `*` is important: without it, `Bash(git diff*)` would also match `git diff-index`.

❗ User-invoked **skills** and custom commands work in `-p` mode: include `/skill-name` in the prompt string and Claude Code expands it before running. Built-in commands that open an interactive dialog, such as `/login`, are not available in `-p` mode. To change a setting from a `-p` invocation, pass `key=value` to `/config`, for example `/config thinking=false`.

Customize the system prompt

Use `--append-system-prompt` to add instructions while keeping Claude Code's default behavior. This example pipes a PR diff to Claude and instructs it to review for security vulnerabilities:

```
gh pr diff "$1" | claude -p \
  --append-system-prompt "You are a security engineer. Review for
vulnerabilities." \
  --output-format json
```

See **system prompt flags** for more options including `--system-prompt` to fully replace the default prompt.

Continue conversations

Use `--continue` to continue the most recent conversation, or `--resume` with a session ID to continue a specific conversation. This example runs a review, then sends follow-up prompts:


```
# First request
claude -p "Review this codebase for performance issues"

# Continue the most recent conversation
claude -p "Now focus on the database queries" --continue
claude -p "Generate a summary of all issues found" --continue
```

If you're running multiple conversations, capture the session ID to resume a specific one:

```
session_id=$(claude -p "Start a review" --output-format json | jq
-r '.session_id')
claude -p "Continue that review" --resume "$session_id"
```

Run both commands from the same directory: session ID lookup is scoped to the current project directory and its git worktrees. See [Resume a session](#) for the full scope rules.

Next steps

- [Agent SDK quickstart](#): build your first agent with Python or TypeScript
- [CLI reference](#): all CLI flags and options
- [GitHub Actions](#): use the Agent SDK in GitHub workflows
- [GitLab CI/CD](#): use the Agent SDK in GitLab pipelines